

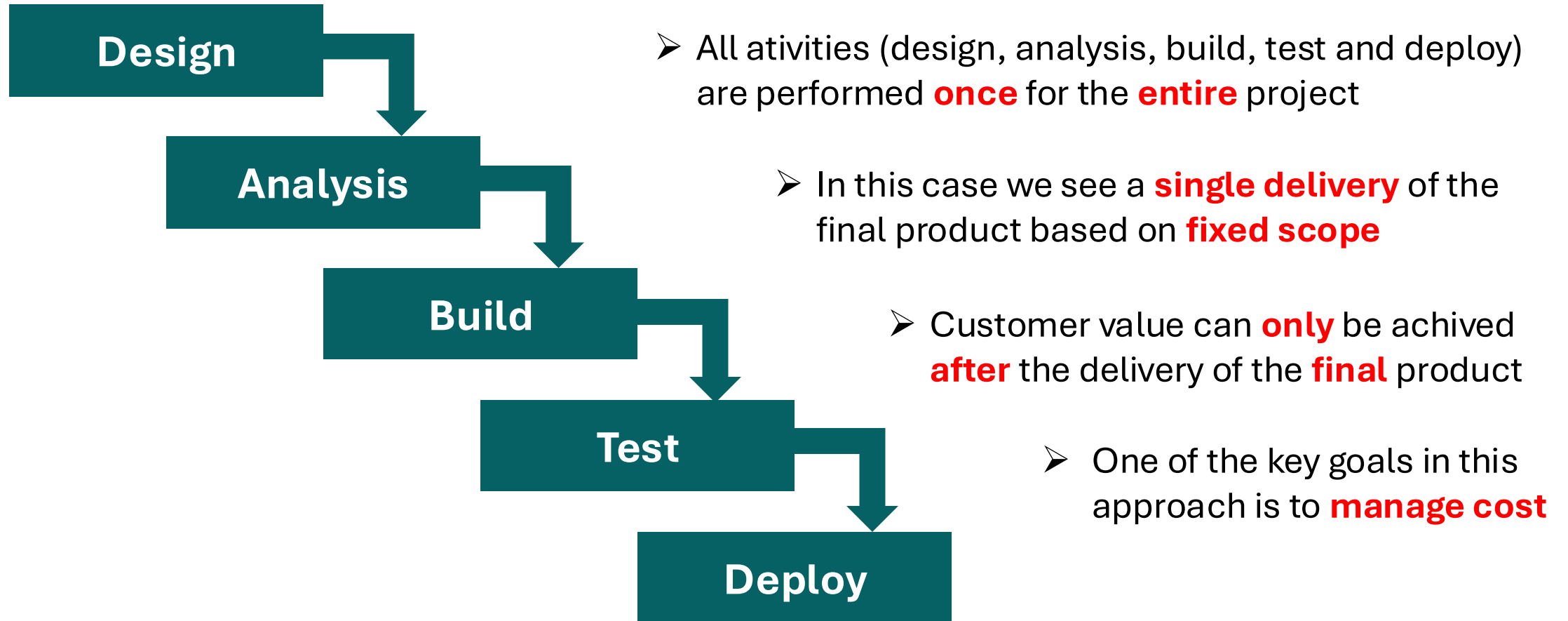
ISP

Lec. #08

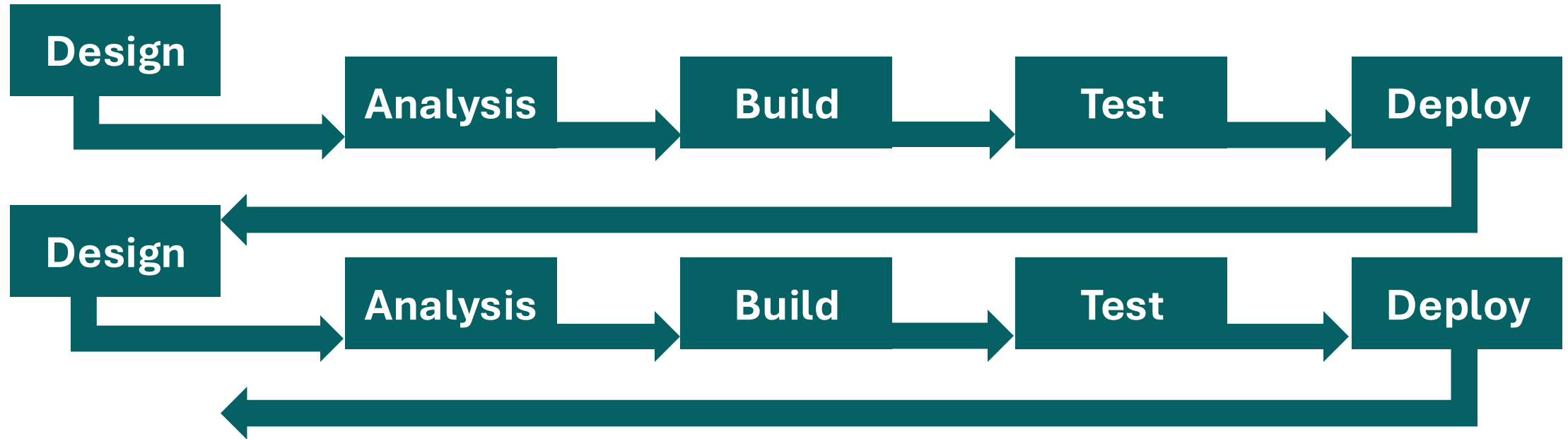
Agile

Scrum

Waterfall

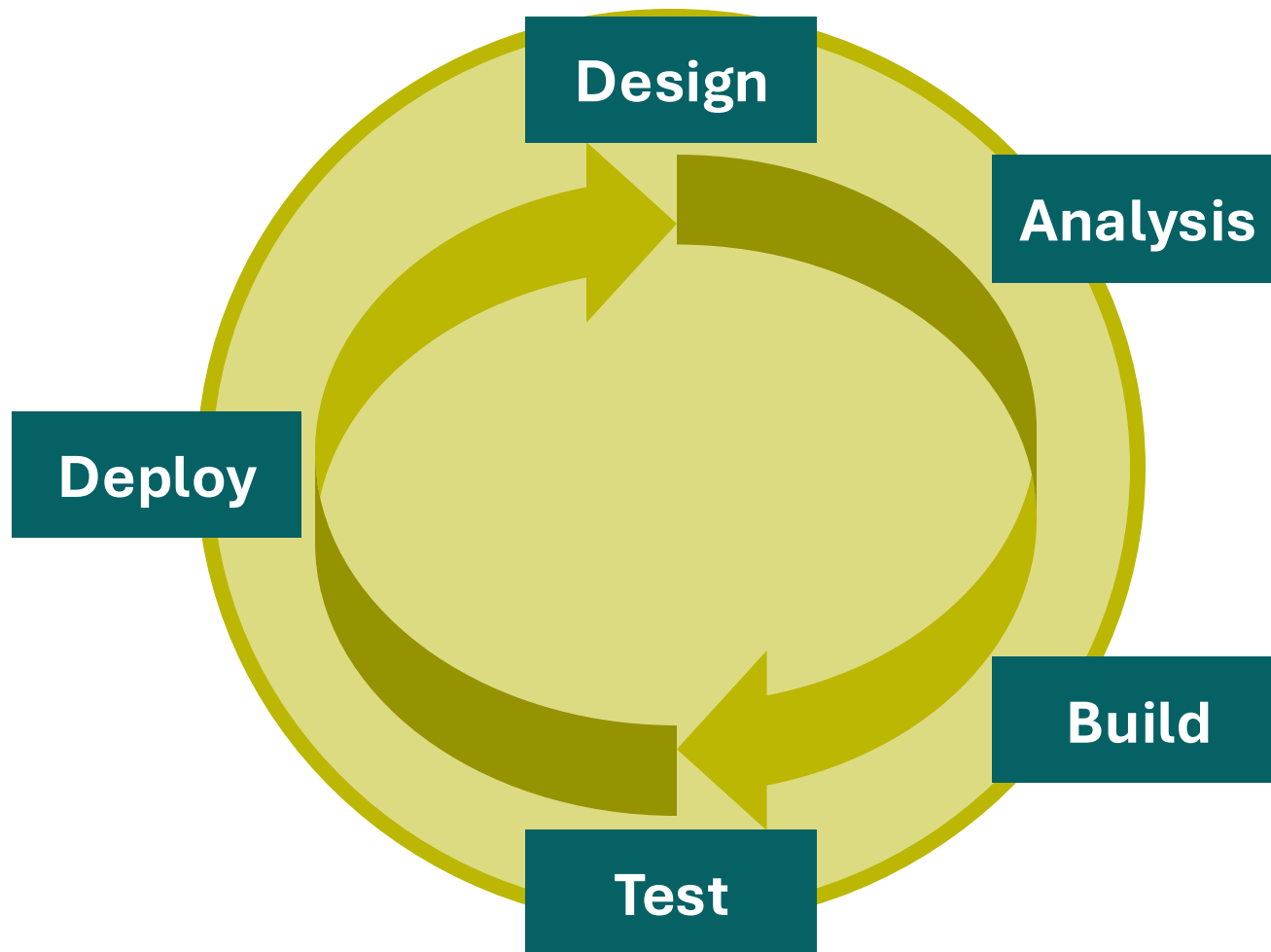


Staged Delivery



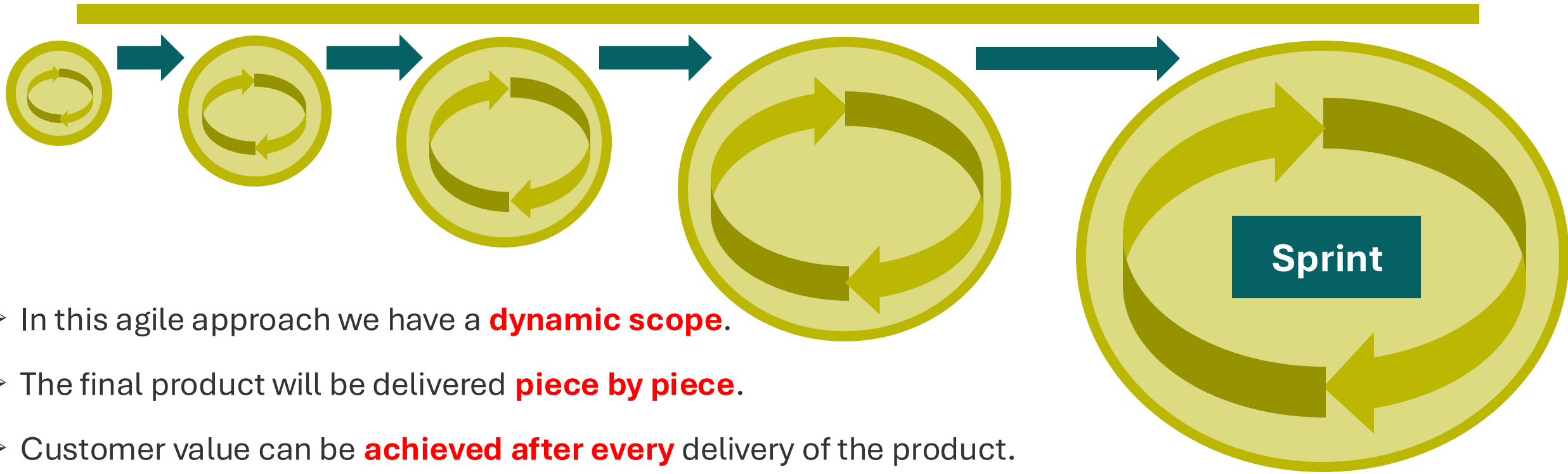
- All activities for **a given stage** (design, analysis, build, test and deploy) are performed **once**.
- Within a **given stage** the scope is **fixed**, but the **total product** is based on a more **dynamic scope**.
- Customer value can be **achieved after every** delivery of the product.
- One of the key goals in this approach is **speed of delivery**.

Spiral



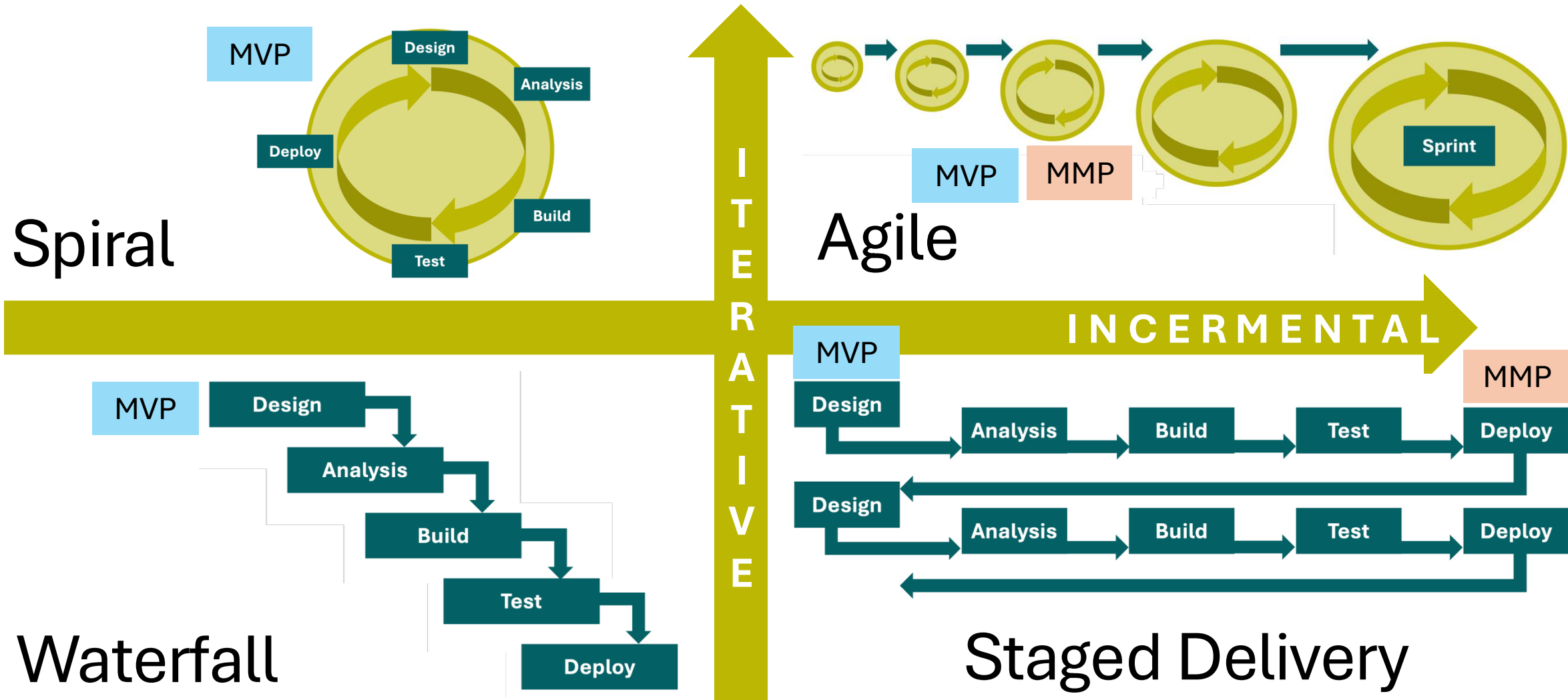
- This is a **single delivery** where the final product is created by using **several iterations**.
- This sequence will be performed repeatedly or iteratively where in every iteration you get **closer to the final**, correct or required, product.
- In many cases this final product is a prototype or model. In this spiral approach we have a **dynamic scope**.
- Customer value can only be achieved **after the delivery of the final** product.
- One of the key goals in this approach is the **correctness of the solution**.

Agile



- In this agile approach we have a **dynamic scope**.
- The final product will be delivered **piece by piece**.
- Customer value can be **achieved after every** delivery of the product.
- This increment is the result of many iterations to develop **small but correct parts**, often called user stories or backlog items, of the product.
- At the end of each increment, often called a **sprint** there is a delivery of an increment of the product.
- One of the key goals in this approach is customer value via **frequent deliveries and customer feedback**.

Iterative & Incremental



MVP Minimum Viable Product

- MVP is the simplest version of a product that has **just enough features** to meet the needs of early adopters and validate the product idea.
- The goal is to learn about your customers' **needs** and gather **feedback** with **minimal investment**.
- The MVP typically focuses on:
 - ✓ Core functionality: Only **the most essential features** necessary to solve a core problem.
 - ✓ Early feedback: **Testing the assumptions** about your product and its market.
 - ✓ Quick iteration: You use this feedback to **make improvements and refine** the product.
- MVP Characteristics:
 - ✓ Simplicity: Includes just enough features to solve a problem or **meet a need**.
 - ✓ Testing and validation: It helps you test the market and **get real-world feedback quickly**.
 - ✓ Low cost: Because it has minimal features, the MVP is **cheaper to develop** compared to a fully featured product.

MMP Minimum Marketable Product

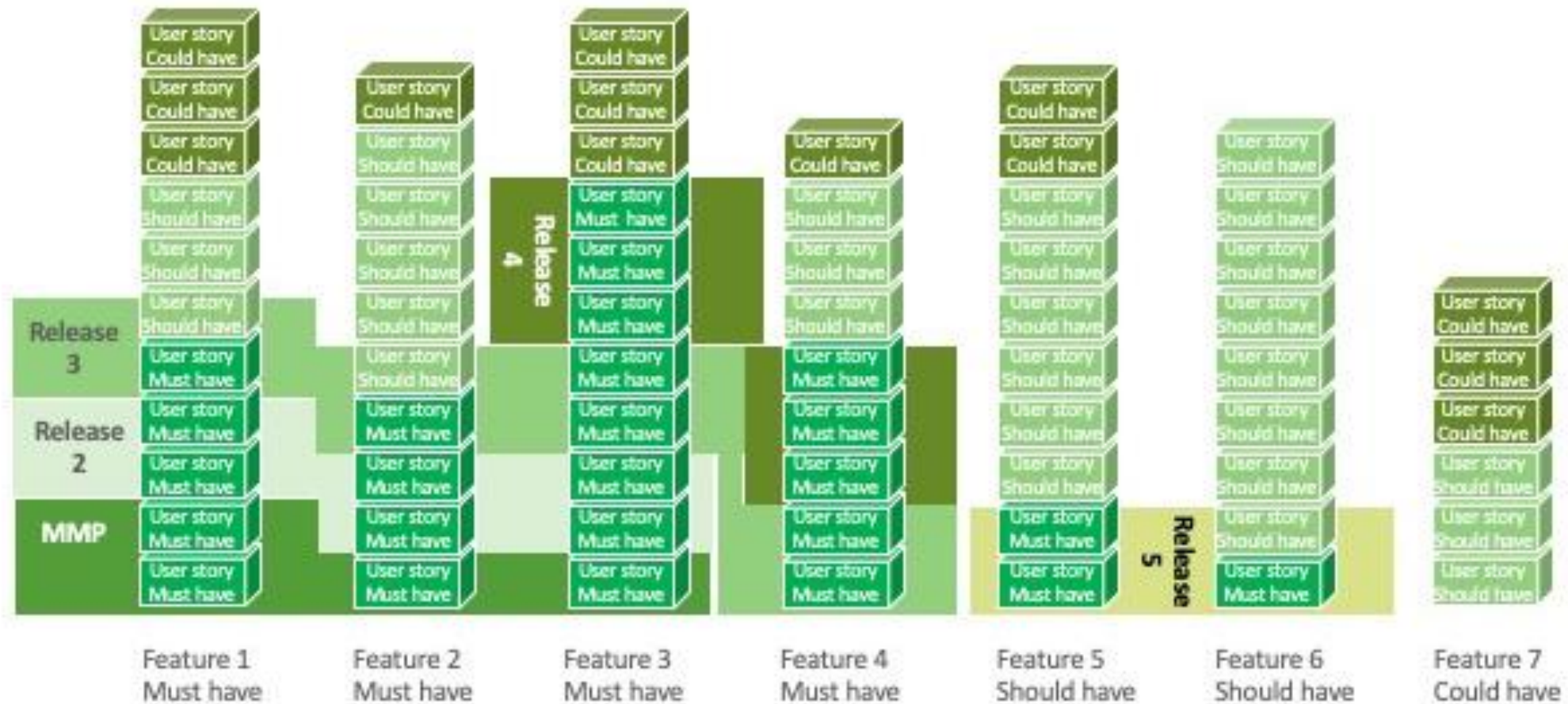
- MMP is a version of the product that is **not only viable** but also capable of **being marketed and generating value** for both users and the business.
- It typically includes a broader set of features that allow the product to meet a **larger market's needs** or expectations and be **more polished** than an MVP.
- The MMP typically focuses on:
 - ✓ Broader functionality: Includes more features that provide **sufficient value to the end user**.
 - ✓ Market-ready: Ready for the market, meaning it can be marketed and sold effectively, even though it **may not have every feature** in the final vision.
 - ✓ Business goals: Focused on creating a product that can **deliver value to users** and help the company grow.
- MMP Characteristics:
 - ✓ Core value proposition: It's a product that can provide a **complete solution**, even if it's not fully developed.
 - ✓ More polished: Has **more features and a better user experience** than an MVP.
 - ✓ Marketable: It's refined enough to be used by a **broader audience and generate revenue or adoption**.

MVP

MMP

Criteria	MVP	MMP
Purpose	Validate ideas and gather feedback .	Deliver a product that's ready for broader use.
Feature Set	Minimal features, just enough to test .	Complete enough to deliver value to users and be marketable.
Audience	Early adopters, testers, or a niche group .	Broader audience, including paying customers .
User Experience	Basic , potentially rough around the edges.	More polished , refined, and user-friendly.
Focus	Learn and iterate.	Serve and satisfy a larger market.
Time to Market	Shorter time, quicker release.	Slightly longer development to ensure market readiness.

Example: Product ABC



Agile Manifesto

<https://agilemanifesto.org/>

We are uncovering better ways of developing
Software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over **process and tools**

Working software over **comprehensive documentation**

Customer collaboration over **contract negotiation**

Responding to change over **following a plan**

That is, while there is value in the items on the right,
we value the items on the left more

Agile Manifesto to Principles

1. Our highest priority is to **satisfy the customer** through early and **continuous delivery of valuable software**.
2. **Welcome changing requirements**, even late in development. Agile processes **harness change** for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must **work together** daily throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team **is face-to-face conversation**.

Agile Manifesto to Principles

1. Our highest priority is to **satisfy the customer** through early and **continuous delivery of valuable software**.

What it means:

The highest priority is to satisfy the customer by delivering working software early and frequently, allowing for continual improvement.

Why it matters:

This principle emphasizes creating value for the customer as early as possible and then iterating on the product to ensure it remains valuable over time.

Agile Manifesto to Principles

2. **Welcome changing requirements**, even late in development. Agile processes **harness change** for the customer's competitive advantage.

What it means:

Agile processes embrace change, even in the late stages of development. Agile is flexible and can adjust to evolving customer needs.

Why it matters:

In the real world, market conditions, user preferences, or business requirements can change. Agile allows for adaptive planning and faster pivoting to accommodate these changes.

Agile Manifesto to Principles

3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

What it means:

Deliver functional software in short, iterative cycles, typically every 1–4 weeks. This could be a new feature, a bug fix, or a product increment.

Why it matters:

Short delivery cycles keep the project moving forward, provide quick feedback, and ensure that the product is always functional, so stakeholders can track progress continuously.

Agile Manifesto to Principles

4. Business people and developers must **work together** daily throughout the project.

What it means:

Close collaboration between business stakeholders (such as product owners or managers) and development teams is crucial.

Why it matters:

By working together regularly, misunderstandings are minimized, and business needs are more easily translated into actionable technical tasks, ensuring the final product aligns with expectations.

Agile Manifesto to Principles

5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.

What it means:

The best results come from individuals who are motivated, trusted, and empowered to make decisions. Developers should be given the right environment and support to thrive.

Why it matters:

Engaged and motivated teams are more productive, creative, and committed to delivering high-quality work. This principle also emphasizes autonomy and accountability.

Agile Manifesto to Principles

6. The most efficient and effective method of conveying information to and within a development team **is face-to-face conversation**.

What it means:

The most effective and efficient method of conveying information is face-to-face communication, especially in collaborative teams.

Why it matters:

Face-to-face interaction ensures clarity and reduces the chances of miscommunication. In remote teams, this principle has been adapted to mean "video calls" or "collaborative tools" that foster real-time communication.

Agile Manifesto to Principles

- 7. **Working software** is the primary **measure** of progress.
- 8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- 9. Continuous attention to **technical excellence** and **good design** enhances agility.
- 10. **Simplicity**--the art of maximizing the amount of work not done--is essential.
- 11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
- 12. At regular intervals, the **team reflects** on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Manifesto to Principles

7. **Working software** is the primary **measure** of progress.

What it means:

Instead of focusing on documentation or deadlines, the progress of an Agile project is measured by the delivery of working software that meets the agreed-upon criteria.

Why it matters:

It's not about following a process or writing lengthy reports; what truly matters is whether the software functions as expected and provides value.

Agile Manifesto to Principles

8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

What it means:

Agile teams should work at a pace that can be maintained indefinitely. Overwork and burnout should be avoided.

Why it matters:

Sustainable work practices lead to healthier teams, reduce stress, and avoid technical debt. Teams that work at a sustainable pace can keep producing high-quality software without risking long-term burnout or diminishing productivity.

Agile Manifesto to Principles

9. Continuous attention to **technical excellence** and **good design** enhances agility.

What it means:

Agile teams strive for technical excellence and simplicity in design. Clean code, best practices, and well-architected systems are prioritized.

Why it matters:

High-quality design and well-maintained code reduce the risk of problems later in development. Good design is easier to maintain, scale, and extend.

Agile Manifesto to Principles

10. **Simplicity**--the art of maximizing the amount of work not done--is essential.

What it means:

Simplicity means focusing on the essential features that deliver value and avoiding unnecessary complexity in design and functionality.

Why it matters:

By removing non-essential features, teams can focus their efforts on what truly matters. Simplicity leads to faster delivery, easier maintenance, and better user experiences.

Agile Manifesto to Principles

11. The best architectures, requirements, and designs emerge from **self-organizing teams**.

What it means:

Teams that are self-organized and have the autonomy to make decisions tend to produce better results, as they have the freedom to adapt and innovate.

Why it matters:

When team members are trusted to organize their work and solve problems collaboratively, the outcomes tend to be more innovative and effective. Autonomy fosters ownership and accountability.

Agile Manifesto to Principles

12. At regular intervals, the **team reflects** on how to become more effective, then tunes and adjusts its behavior accordingly.

What it means:

At regular intervals, teams should reflect on how to improve their work processes and make adjustments accordingly. This is typically done in "retrospectives."

Why it matters:

Continuous improvement is a core tenet of Agile. By regularly reflecting on what's working and what's not, teams can optimize their processes, avoid mistakes, and enhance efficiency over time.

Agile Characteristics

#	Characteristic	Description
1	Iterative & Incremental	Development in small cycles with frequent releases.
2	Customer-Centric	Continuous collaboration with customers for feedback and value.
3	Adaptability	Ability to change direction based on new insights or feedback.
4	Collaboration	Regular and open communication between cross-functional teams.
5	Simplicity	Focusing on essential features, avoiding unnecessary complexity.
6	Continuous Improvement	Regular retrospectives for process and product refinement.
7	Self-Organizing Teams	Empowering teams to make decisions independently.
8	Sustained Pace	Maintaining a consistent, manageable work pace for long-term success.
9	Frequent Delivery	Regular releases of working software to get quick feedback.
10	Focus on People	Valuing human communication and collaboration over strict processes.
11	Working Software	Prioritizing functional software over extensive documentation.
12	Transparency	Ensuring visibility into progress for all stakeholders.

Agile Pros & Cons

Agile methods are being widely accepted in the software world recently. However, this method may not always be suitable for all products

Advantages

- Flexibility and Adaptability
- Customer Satisfaction
- Faster Time to Market
- Improved Collaboration and Communication
- Higher Quality
- Employee Engagement and Motivation
- Risk Management

Agile Pros & Cons

Agile methods are being widely accepted in the software world recently. However, this method may not always be suitable for all products

Disadvantages

- Scope Creep
- Requires High Customer Involvement
- Challenges in Large Teams or Projects
- Limited Documentation
- Requires Skilled Teams
- Difficulty in Predicting Long-Term Outcomes
- Overemphasis on Process
- Resistance to Change

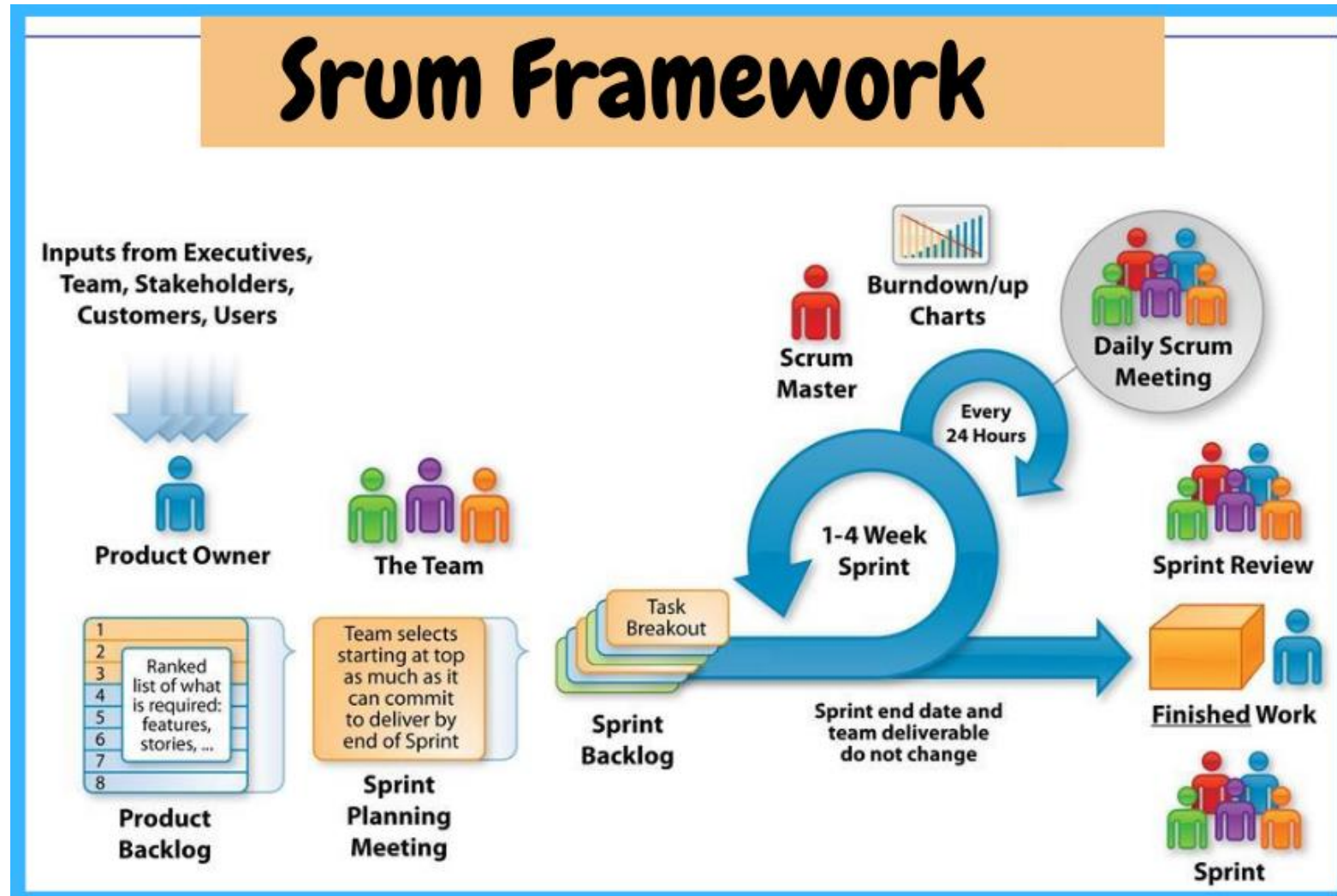
Agile Frameworks

1. Scrum
2. Kanban
3. XP (Extreme Programming)
4. Lean
5. FDD (Feature-Driven Development)
6. Crystal
7. DSDM (Dynamic System Development Support)
8. SAFe (Scaled Agile Framework)
9. DAD (Disciplined Agile Delivery)
10. AUP (Agile Unified Process)

Scrum

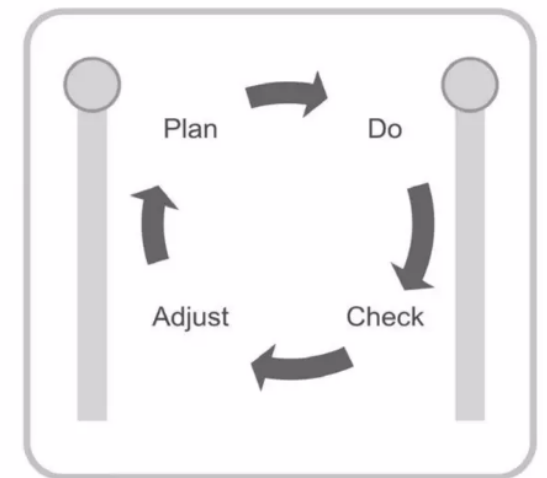
- Scrum is a framework for **develeloping and sustaining** complex products
- Ken Schwaber and Jeff Sutherland developed Scrum
- Scrum is a framework within which **people can addrees complex adaptive problems**, while productively and creatively delivering products of the highest possible value
- Scrum is an agile process
- Business/Customer **sets the priority**
- Teams self organize to determine the **best way to deliver the highest** priority features
- Scalable to distributed, large and long projects
- Scrum is a specific agile process framework that defines **the practices required** to be followed

Scrum



Sprint

- Scrum is an iterative methodology, and Scrum name for iteration is **Sprint**
- Iterations are single development cycle where each agile teams defines, builds, integrates and test **the user stories** from their iteration backlog
- The goal is to deliver working software **at the end of each iteration**
- Each iteration is of **same length** running back to back



Sprint

- In the diagram below you can see a Scrum project development split into 24 iterations for a full year duration

Sprint 0				Sprint 1		Sprint 2		Sprint 3		...	Sprint 24	
W1	W2	W3	W4	W5	W6	W7	W8	W9	W10		W51	W52

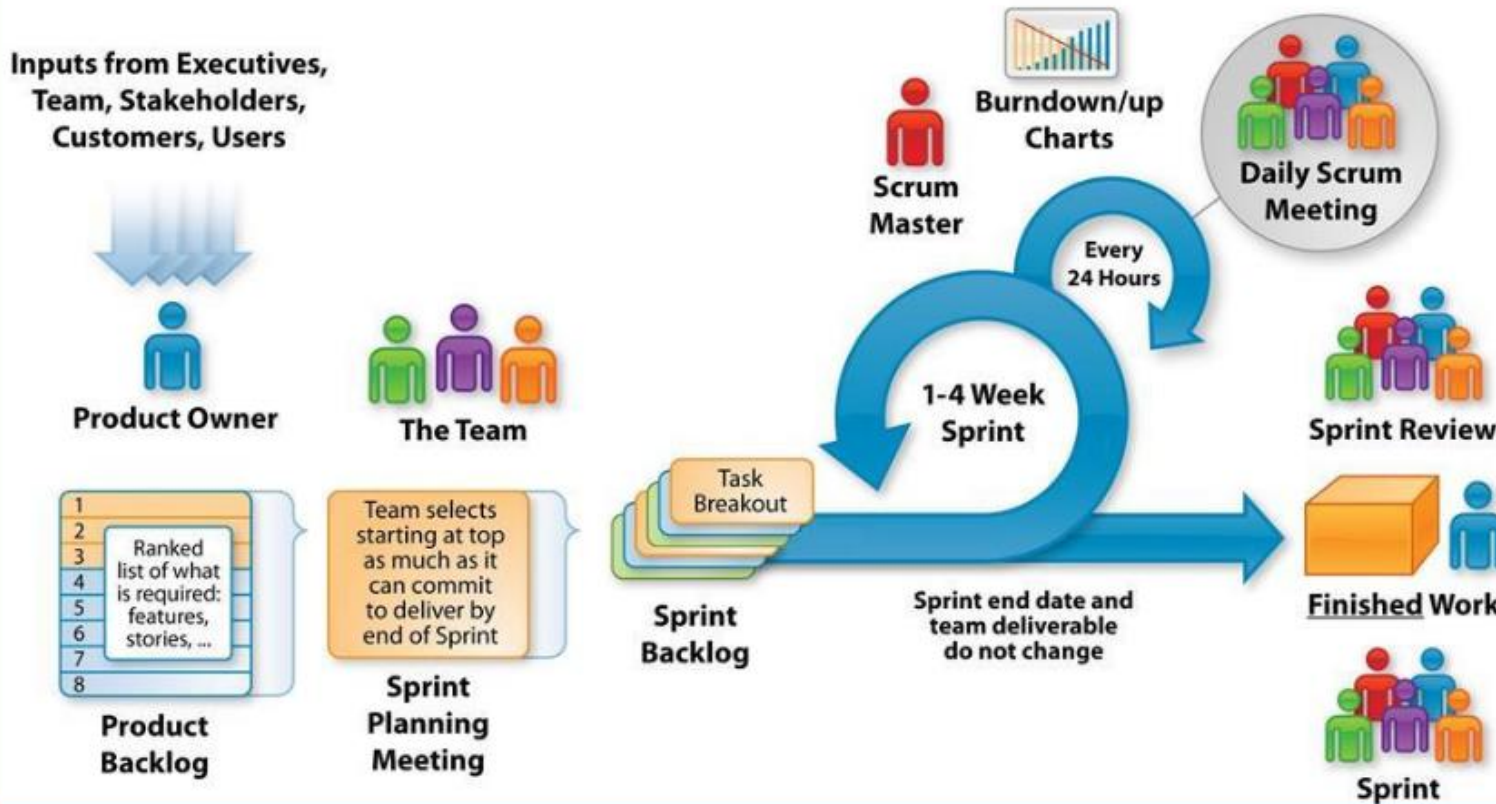
- Sprint 0 is a **special iteration** performed at the **beginning of a project** and forms the framework upon which each subsequent iteration builds. The duration of this iteration will vary depending on **the complexity and existing knowledge of the desired capability**.
- During this special Spring perform enough architecture and system design in order to **start development and get the first Spring started**.

Sprint

- Additional architecture and system engineering will be performed as necessary **during subsequent iterations**
- After Sprint 0 all other Sprints must have the same length. This length could be between 1 and 4 weeks. The recommended length is **2 weeks**
- At the beginning of the Sprint, the team has a clear **commitment on the scope**, and the scope for current Sprint cannot be changed during the Sprint
- Short iterations enable Product Owners to get client feedback faster, developer estimates faster, and faster updates to plans, thereby **shortening the whole feedback cycle**.

Scrum Roles

Scrum Framework



1. Product Owner
2. Scrum Master
3. Development Team

1. Product Owner

- ✓ Defines the features of the product
- ✓ Decide on release date and content
- ✓ Be responsible for the profitability of the product (ROI)
- ✓ Prioritize features according to market value
- ✓ Adjust features and priority every iteration, as needed
- ✓ Accept or reject work results

2. Scrum Master

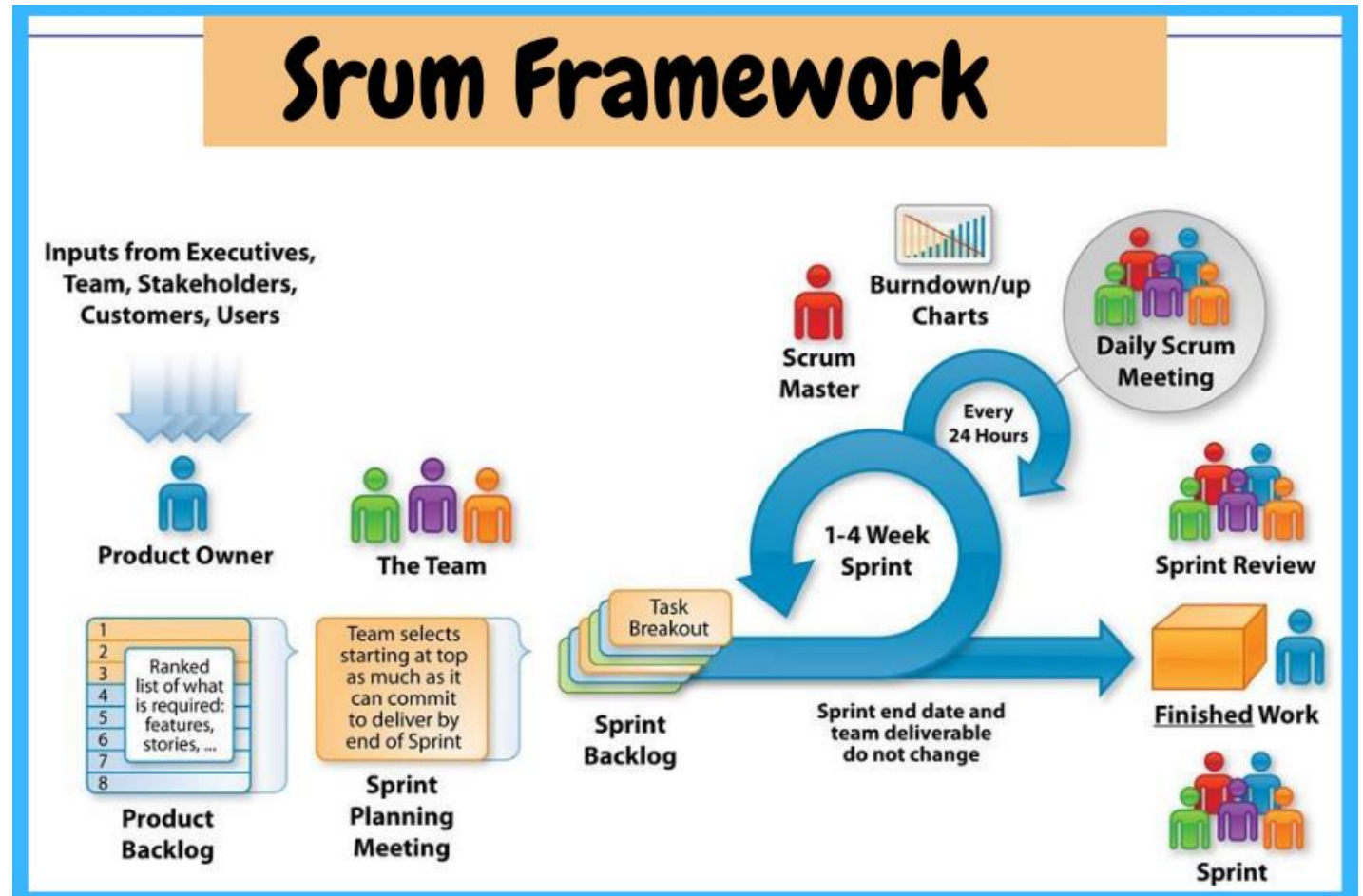
- ✓ Ensure that the team is fully functional and productive
- ✓ Enable close cooperation across all roles and functions
- ✓ Remove barriers
- ✓ Shield the team from external interferences during the Sprint
- ✓ Ensure that the process is followed, including issuing invitations to Daily Scrum, Sprint Review, and Sprint Planning meetings

3. Development Team

- ✓ Is cross-functional (Skills in testing, coding, etc.)
- ✓ Selects the Sprint goal and specific work results
- ✓ Has right to do everything within the boundaries of the project guidelines to reach the Sprint goal
- ✓ Organizes itself and its work
- ✓ Demo work results to Product Owner

Scrum Events

1. Backlog Grooming
2. Sprint Planning
3. Daily Scrum Meeting
4. Sprint Review
5. Sprint Retrospective



1. Backlog Grooming

- **Purpose:** To ensure that the **Product Backlog** is up-to-date, organized, and prioritized, and that the user stories or tasks are well-defined and ready for the upcoming sprints.
- **Key Activities:**
 - ✓ Prioritize items in the backlog based on business value, customer feedback, or technical needs.
 - ✓ Break down larger user stories (epics) into smaller, manageable tasks (stories).
 - ✓ Clarify any ambiguities or questions about backlog items.
 - ✓ Add or remove items based on new insights or changing requirements.
- **Frequency:** Typically occurs **throughout the sprint** and may be a recurring activity (e.g., weekly or as needed).
- **Participants:** The **Product Owner** (who owns the backlog), **Scrum Master**, and **Development Team** (for technical input).

2. Sprint Planning

- **Purpose:** To define the work that will be completed during the upcoming **Sprint** and create a plan to achieve the Sprint Goal.
- **Key Activities:**
 - ✓ **Product Owner** presents the **highest-priority backlog items** to the team.
 - ✓ **Development Team** selects the tasks they will commit to completing in the sprint, based on their capacity and the sprint goal.
 - ✓ The **team** defines **how** they will implement the selected backlog items (task breakdown).
 - ✓ Agree on a clear **Sprint Goal** (the primary objective for the sprint).
- **Frequency:** Happens at the **beginning of each Sprint**.
- **Participants:** **Product Owner, Scrum Master, Development Team.**
- **Outcome:** A **Sprint Backlog** that details the tasks and the Sprint Goal.

3. Daily Scrum Meeting

- **Purpose:** To synchronize the team's work, share progress, and identify any blockers or issues that could impede the team from reaching the Sprint Goal.
- **Key Activities:**
 - ✓ Each team member answers three questions:
 - What did I accomplish yesterday?
 - What will I work on today?
 - Are there any blockers or challenges in my way?
 - ✓ The team discusses progress towards the **Sprint Goal and identifies potential issues early.**
- **Frequency:** Happens **every day** of the Sprint, usually at the same time and place.
- **Duration:** **15 minutes (time-boxed).**
- **Participants:** The **Development Team**, **Scrum Master** (who facilitates), and **Product Owner** (optional but often attends to stay informed).
- **Outcome:** Clear understanding of daily progress and any issues to address.

4. Sprint Review

- **Purpose:** To demonstrate the work completed during the Sprint to stakeholders and obtain feedback for improvement.
- **Key Activities:**
 - ✓ **Development Team** presents the work (increment) they've completed, demonstrating new features or functionality.
 - ✓ **Product Owner** reviews the work and compares it to the **Sprint Goal** and the **Definition of Done**.
 - ✓ **Stakeholders** (internal or external) provide feedback, which can influence **the Product Backlog** and the next Sprint.
- **Frequency:** Happens at the **end of each Sprint**, typically after the last day of the Sprint.
- **Duration:** Usually around **1-2 hours** depending on Sprint length.
- **Participants:** Scrum Team, Stakeholders.
- **Outcome:** Feedback is gathered, and **Product Backlog** may be adjusted based on the results and insights.

5. Sprint Retrospective

- **Purpose:** To reflect on the Sprint, identify what went well, what didn't, and define actionable improvements to enhance team performance.
- **Key Activities:**
 - ✓ The team discusses **what went well** during the Sprint, recognizing successes.
 - ✓ They identify **what didn't go well**, such as bottlenecks, obstacles, or communication issues.
 - ✓ The team suggests and agrees on **actions or process changes** to improve the next Sprint.
 - ✓ The **Scrum Master** helps facilitate the retrospective, ensuring it's focused on improvement.
- **Frequency:** Happens at the **end of each Sprint**, after the Sprint Review.
- **Duration:** Typically **1-1.5 hours** (depending on the Sprint length).
- **Participants:** The **Scrum Team** (Product Owner, Scrum Master, Development Team).
- **Outcome:** Actionable **improvements that help the team work** more effectively in future Sprints.

Scrum Artifacts

1. Product Backlog

- **Definition:** The **Product Backlog** is an ordered list of all the features, functionalities, enhancements, bug fixes, technical improvements, and other work items needed to complete a project. It's essentially a **living document** that evolves over time based on stakeholder feedback, business priorities, and changing market conditions. The Product Backlog is continuously refined and adjusted, and it represents the overall scope of the product.
- **Owner:** The **Product Owner** is responsible for managing the Product Backlog, ensuring it is properly prioritized, and keeping it aligned with business goals.
- **Content:** It can contain user stories, technical tasks, bugs, features, research items, or any work needed to build the product.
- **Characteristics:**
 - ✓ **Prioritized:** Items are ranked in order of importance, with the most valuable items appearing at the top.
 - ✓ **Dynamic:** It evolves over time. New items can be added, and existing items can be refined, split, or removed as the project progresses.
 - ✓ **Granularity:** Items near the top of the backlog are generally more detailed, while those further down may be less defined.

Scrum Artifacts

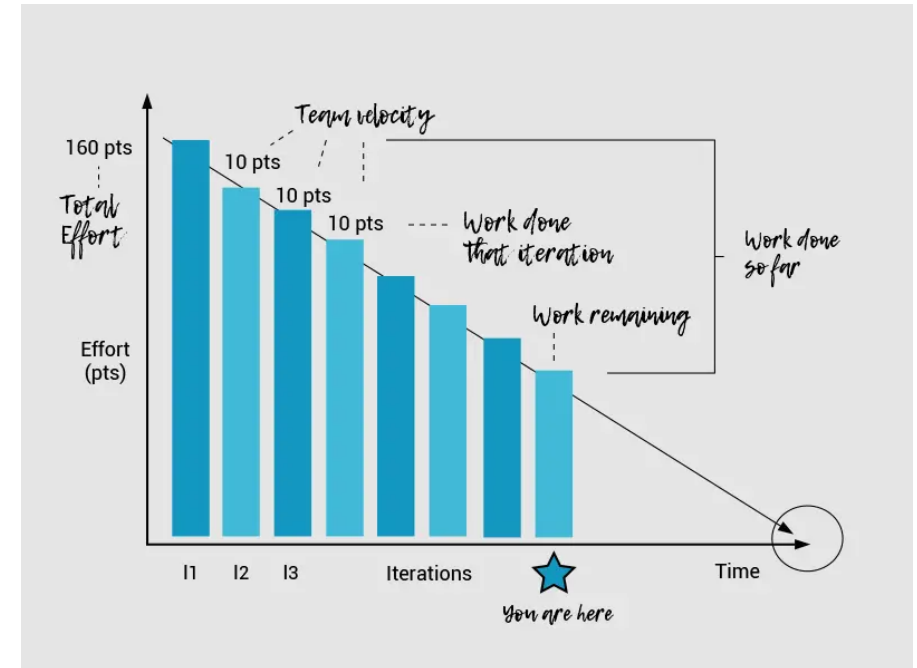
2. Sprint Backlog

- **Definition:** The **Sprint Backlog** is a **subset** of the Product Backlog that the Development Team commits to working on during a specific **Sprint**. It is a detailed, actionable plan that breaks down the **highest-priority items** from the Product Backlog into smaller tasks that can be completed during the Sprint. It is owned and managed by the Development Team, who can modify it as needed throughout the Sprint.
- **Owner:** The **Development Team** owns and manages the Sprint Backlog.
- **Content:** The Sprint Backlog includes:
 - ✓ A selection of items from the **Product Backlog** (usually user stories).
 - ✓ A detailed list of tasks required to complete the selected work.
 - ✓ The **Sprint Goal**, which is a clear objective for the Sprint.
- **Characteristics:**
 - ✓ **Focused:** It contains only the work the team has committed to completing within the current Sprint.
 - ✓ **Dynamic:** It can evolve during the Sprint if the team discovers more work needs to be done, but it should not expand beyond the Sprint's capacity.
 - ✓ **Actionable:** Items in the Sprint Backlog are broken down into **tasks** that can be worked on during the Sprint.

Scrum Artifacts

3. Burndown Chart

- **Definition:** A **Burndown Chart** is a graphical representation of the amount of work remaining in a Sprint or a project over time. It shows **how quickly the team** is burning down the remaining tasks or work items, and it provides a visual indication of whether the team is on track to complete the work by the end of the Sprint or release.
- **Owner: Scrum Master** typically creates and tracks the Burndown Chart, but it's a tool used by the entire Scrum Team to visualize progress.



➤ Characteristics:

- ✓ **Visual Tracking:** It provides a clear visual representation of how much work remains to be done.
- ✓ **Forecasting:** By tracking the slope of the Burndown Chart, you can forecast whether the team will complete the work by the end of the Sprint.
- ✓ **Adjustments:** If **the line flattens (indicating a slowdown)**, the team might adjust its approach or work more efficiently to catch up.

Scrum User Stories

- User Story (Shortly US) : It is a short, simple description of a feature told from the perspective of the person who **desires the new capability**, usually a user or customer of the system
- Acceptance criteria : Each User Story also has Acceptance Criterion defined, so that **correctness of implementation** of the user story is confirmed by passing the **Acceptance Test** that is based on the Acceptance Criteria
- INVEST helps to remember a widely accepted set of criteria, or checklist, to assess the quality of a user story
 - ✓ I : Independent
 - ✓ N : Negotiable
 - ✓ V : Valuable
 - ✓ E : Estimable
 - ✓ S : Small
 - ✓ T : Testable
- User story structure: As a **<user role>**, I want **<activity>** so that **<business value>**
 - ✓ User Role : Description of person, device or system doing action
 - ✓ Activity : It is what they can do with the system
 - ✓ Business Value : It is why they want to do the activity

Scrum User Stories

- Epic : a user story that covers **large amounts of functionality**. Because an epic is generally too large for an agile team to complete in one iteration, it is split into multiple smaller user stories before it is worked on
- User Stories are managed in the **Product Backlog**. The User Stories are ordered according to priority
- Examples: Epic1
 - ✓ As a user, I can backup my entire hard drive, so that I can recover data after its loss
- This epic can be broken in a set of user stories
 - US1 As a user, I can specify files or folders to backup based on file size, created date and modified dates, so that . . .
 - US12 As a user, I can indicate folders not to backup so that my backup drive is not filled up with things I do not need saved, so that . . .
- For every sprint, the most prioritized and hence **more granular** user stories are taken into the sprint backlog.
- If a user story is to be added to the product backlog, its priority is first determined, and it is placed according to its **place as per the priority**.
- The user stories can be **reprioritized at any time**. It is also possible to remove any of the user stories if required

Scrum Estimation

- In Scrum Projects, Estimation is done by the **entire team during Sprint Planning Meeting**. The objective of the Estimation would be to consider the User Stories for the Sprint by Priority and by the Ability of the team to deliver **during the Time Box of the Sprint**
- Product Owner ensures that the **prioritized User Stories are clear**, can be subjected to estimation, and they are brought to the beginning of the Product Backlog. The size of the **Product Increment** is estimated in terms of User Story Points
- There are **several types of scales** that are used in Scrum Estimation
- Example
 - ✓ Numeric Sizing (1 through 10)
 - ✓ T-shirt Sizes (XS, S, M, L, XL, XXL, XXXL)
 - ✓ Fibonacci Sequence (1, 2, 3, 5, 8, 13, 21, 34, etc.)
 - ✓ Dog Breeds (Chihuahua, . . . , Great Dane)

Scrum Estimation

- Playing Poker technique is used to estimate the User Stories in Story Points (SP) with the next possible values (Fibonacci numbers): 1, 2, 3, 5, 8, 13, 21. By using cards, each user story is estimated by the entire team. **The estimation for a user story is made in secret by the team**, then the cards are revealed at the same time and the team members with higher and lower estimates must presents **their reasons for the estimation**. This estimation process for the user story is repeated until the entire team have the same estimation for the user story
- Story Points are relative values, a story that estimated with 2 story points should be twice as much as items that are estimate with 1 story point. **Story points represent the effort to develop a story and this includes everything that can affect the effort including amount, complexity and risks associated with this work**
- Hint: each User Story must
 - ✓ Be created by using the above template
 - ✓ Respect INVEST rule
 - ✓ Have a set of acceptance criteria
 - ✓ Be estimated by the team in Story Points by using Playing Poker technique

Scrum Tools

- Scrum Tools facilitate planning and tracking for Scrum projects. They provide a single place for managing the product backlog, sprint backlog, planning and tracking Sprints, displaying Burndown charts, conducting daily Scrum meetings, and conducting Scrum Retrospectives
- Some available scrum tools in Market
 1. Jira
 2. Mingle
 3. Daily Scrum
 4. Scrum Factory
- Agile in general, Scrum in specific does not mean there is no documentation work. The Scrum Artifacts are defined, Scrum Planning and Tracking are well established
- Scrum Tools facilitate in capturing and tracking information regarding the Scrum projects. The choice of the tool depends on the feature required by the organization, in addition to the needs for any other tool

Jira <https://www.atlassian.com/software/jira>

The screenshot displays the Jira homepage with a navigation bar at the top containing links for Features, Solutions, Product guide, Templates, and Pricing, along with a 'Get it free' button and a 'Sign in' link. The main heading reads 'Great outcomes start with Jira' with the tagline 'The only project management tool you need to plan and track work across every team.' Below this is a sign-up section with a 'Work email' input field, a 'Sign up' button, and options to continue with Google or Microsoft. The bottom section features five use case cards: 'Software Development' (Product & Issue Tracking), 'Marketing' (Plan & Launch Campaigns), 'IT' (Plan & Track IT Projects), 'Design' (Build Creative Workflows), and 'Operations' (Create Custom Processes). Each card includes a visual representation of its workflow, such as a Kanban board for Marketing or a task list for Operations.

Software Development

PRODUCT & ISSUE TRACKING

IN PROGRESS

Optimize experience for mobile web

FEEDBACK

NUC-335 2

Bump version for new API

FORMS

NUC-336 5

Adapt web app no new payments provider

Marketing

PLAN & LAUNCH CAMPAIGNS

SUN	MON
27	28
FIN-23 Workshop with...	
4	5
FIN-45 Do something useful	
11	12
FIN-56 Design spec kick-off	

IT

PLAN & TRACK IT PROJECTS

✓ All changes saved

Licensing and billing form

Please make sure you submit your request help us prioritize your asks within our roadmap

Summary

Description

Due date

Design

BUILD CREATIVE WORKFLOWS

EXP-21 / EXP-1

Implement integrations into

Attach Create subtask

Designs (1)

Figma

Operations

CREATE CUSTOM PROCESSES

Assignee	Due
Alana Song	17 Aug
Fran Perez	29 Sep
Andres Ramos	07 Nov
Amar Sundaram	01 Oct
Fran Perez	16 Nov
Alana Song	12 Oct
Molly Clark	18 Sep
Eva Lien	28 Oct

Agile Frameworks

#		Focus on	Best for
1	Scrum	Iterative development in sprints	Teams that need structured roles and ceremonies
2	Kanban	Continuous flow, visualizing work	Ongoing work with a need for flexibility and flow
3	XP	Software quality, technical excellence	Teams requiring high code quality and engineering practices
4	Lean	Eliminating waste, optimizing flow	Teams focused on process improvement and efficiency
5	FDD	Feature-centric development	Large projects or those focused on feature delivery
6	Cystral	Human-centric, adaptable framework	Small teams or projects with high emphasis on communication
7	DSDM	Full project lifecycle, fixed time and cost	Projects with well-defined constraints (time, cost)
8	SAFe	Scaling Agile to the enterprise level	Large organizations or multiple teams
9	DAD	Process decision framework	Organizations needing guidance on Agile practices
10	AUP	Simplified version of Rational Unified Process	Teams wanting a formalized, structured approach with Agile principles

Q&A

Email: kundjanasith.th@ku.th